Systematic Mutation and Black-Box Probing for SQL Injection WAF Bypass: A Live Testbed and Family-Level Analysis

Tran Xuan Quyen
School of Info. & Comm. Technology
HUST
Hanoi, Vietnam
quyen.tx252593m@sis.hust.edu.vn

Co Huy Dung
School of Info. & Comm. Technology
HUST
Hanoi, Vietnam
dung.ch252592m@sis.hust.edu.vn

Nguyen Duc Toan
School of Info. & Comm. Technology
HUST
Hanoi, Vietnam
toan.nguyenduc1@hust.edu.vn

***Abstract*—Attackers routinely evade Web Application Firewalls (WAFs) by mutating SQL injection (SQLi) payloads. Prior studies report high bypass rates, yet these aggregate figures often conflate heterogeneous operators and depend on search biases. Consequently, defenders need reproducible, family-resolved measurements tied to explicit labels.**

This paper presents an offensive methodology that generates discrete SQLi variants through a dialect seed, deployment-specific augmentations, and learned transform rules. We issued black-box HTTP probes against a live AWS WAF-fronted application, collecting per-attempt JSONL logs. In our 21,000-variant deployment experiment, our operational classifier identified a 47.0% clean bypass rate (HTTP 200 OK), with an additional 26.6% of attempts inducing server-side errors (HTTP 500) that confirm WAF evasion but not successful query execution. However, family-level analysis reveals extreme heterogeneity: URL multi-encoding and null-byte cohorts achieved bypass rates exceeding 99%, while space-replacement and comment-block mutations were systematically blocked. This indicates that policy stress concentrates in specific syntactic strategies.

As a secondary control, we applied repeated RFC percent-decoding and scored strings with a TF-IDF logistic-regression surrogate. On a 20-request pilot, accuracy remained at 42.9% for both raw and canonicalized inputs, suggesting that naive decoding neither closes the live bypass gap nor rescues a simple offline model. All reported rates are conditional on the specific experimental configuration; the core contribution lies in the explicit methodology, decomposed statistics, and reproducible artifact.

***Index Terms*—SQL injection, web application firewall, black-box attack, adversarial mutation, bypass measurement, variant families**

I. Introduction

Web applications remain the primary interface for online services, with Web Application Firewalls (WAFs) widely deployed to block injection attacks [9]. From an attacker's perspective, the key challenge is navigating a specific deployment's concrete rules and parsers to achieve a security-relevant application outcome. Signatures can be evaded by semantic-preserving rewrites, and parsers can disagree on decoding and parameter binding. While these phenomena are well known, measuring the conditions and specific mutation structures under which bypasses occur remains a scientific challenge.

This paper contributes an operational, attack-centric evaluation. We systematically mutate SQLi fragments, probe a production-style AWS WAF deployment, and analyze outcomes at both aggregate and variant-family levels. We release our implementation as a reproducible artifact to provide red and blue teams with empirical distributions for prioritization and regression testing.

A. Problem Statement

SQLi evasion combines representation-level mutation—altering substrings or encodings seen by a pattern matcher—and interpretation-level divergence, where the WAF and application decode the same sequence differently [2].

Prior adversarial methods demonstrate that the attack space is combinatorial. Systems like SSQLi [1], GSQLi [7], and WAF-A-MoLE [6] achieve high bypass rates by searching this space. However, success is rarely uniform across operators. Published rates seldom decompose by operator family on live managed WAFs, obscuring which mutations survive. Furthermore, while mutating HTTP structure induces parsing discrepancies [2], we focus on URL-visible injection with deterministic seed remapping. This minimizes parameter-name mismatch, providing a controlled setting to study representation-level evasion against a fixed policy.

B. Objectives and Research Questions

We structure our empirical evaluation around three core questions:

- **RQ1 (Scale and semantics).** Under an explicit bypass definition, what fraction of attempts succeed against our AWS WAF lab, and how does this relate to unique payload cardinality?
- **RQ2 (Structure).** How much bypass mass concentrates in specific mutation families, and which are systematically suppressed?

In addressing these questions, we formalize a black-box attack pipeline for managed WAF assessment. We provide an empirical characterization of a live AWS WAF run, detailing unique payload counts and family-level bypass rates that reveal

sharp heterogeneity. Finally, we include a secondary control showing that basic canonicalization does not immediately rescue simple classifiers.

C. Paper Organization

Section II covers related work. Section III details our methodology. Section IV presents the evaluation answering RQ1–RQ3. Section V concludes the paper.

II. Background and Related Work

Offensive SQLi evaluation bridges adversarial search against detectors, HTTP semantics, and operational security measurement. We highlight strands that motivate our focus on family-resolved reporting.

A. Mutation-Based SQLi Attacks

Adversarial methods search for inputs that preserve malicious intent while evading detectors. Reinforcement learning approaches like SSQLi [1], GAN-based methods like GSQLi [7], and semantics-preserving fuzzing like WAF-A-MoLE [6] achieve high bypass rates. The underlying attack space is combinatorial, involving comment insertions, encodings, and dialect-specific spellings that interact non-monotonically with tokenized defenses. Consequently, the relevant metric is not just the aggregate bypass rate, but which specific compositions survive, as this informs both attacker and defender strategies.

B. Parsing Discrepancies and Defense Evaluation

When components disagree on decoding order or structural boundaries, attackers can satisfy edge predicates while delivering malicious semantics [2]. WAFFLED demonstrated this across major WAFs. Our experiments stabilize parameter binding via remapped seeds to isolate representation-level variance.

Hybrid defenses aim to raise evasion costs through better data [8], shadow models [5], or preprocessing [4]. However, evaluating these defenses requires paired offensive benchmarks. Without logs tying attempts to mutation families, it is difficult to determine if a mitigation removes high-yield operators or merely reshuffles the search order. Our work supplies a log-grounded baseline for such comparisons.

C. Metrics and Confounding Threats

Black-box evaluation relies on observable HTTP outcomes. Since different probes map the same vulnerability to different observables, any operational definition of a bypass embeds epistemic commitments. Transparent scripting and published logs mitigate this issue, though they do not eliminate it entirely.

III. Methodology

This section details our adversarial evaluation methodology, including the experimental environment, adversary capabilities, and the attack pipeline. Artifacts are available under `paper/experiment`.

Seeds → augmentations & rules → 2nd-layer transforms → probes → logs

Fig. 1. Offensive evaluation workflow; policies configurable in `config.json`.

A. Experimental Environment

We deployed the study on AWS, sending traffic through an AWS WAF Web ACL in `us-east-1` before reaching an EC2-hosted application. This mirrors a production environment where the backend only sees what the WAF forwards.

The Web ACL (`test-waf`) features three sequential rules:

1) **`Block-SQLi-Strict-Pattern`:** Targets simple tautologies (e.g., `1=1`) evaluated across all query arguments after decoding and normalization.
2) **`AWS-AWSManagedRulesSQLiRuleSet`:** Amazon's managed SQLi bundle, set to block any hit, providing broad vendor coverage across the request structure.
3) **`Final-Optimized-SQLi-Rule`:** A custom regular expression stage targeting specific lab leftovers, such as hex-flavored encodings and null-byte obfuscation.

The endpoint is a custom web application on a `t3.micro` EC2 instance. We drove probes manually via scripts without relying on generic scanners like sqlmap. Outcomes were observed directly from HTTP responses: a `403` indicated a block, while a `200` indicated the request reached the application.

B. Adversary Model and Attack Pipeline

We model a network adversary capable of sending HTTP/1.1 requests, observing responses, and adapting mutations based on logs. The adversary lacks white-box access, aligning with typical red-team constraints [1], [6].

The end-to-end pipeline operates as follows:

$$\begin{aligned} \mathcal{M} : \text{Seeds} \to \text{Augmentations} \to \text{Transforms} \to \mathcal{V}, \\ \mathcal{V} \xrightarrow{\text{HTTP}} \text{WAF} \to \text{App} \to \text{Label}. \end{aligned} \tag{1}$$

The driver serializes each variant in $\mathcal{V}$ as a GET request, logs the response, and outputs a JSON object.

C. Staged Mutation Generation

To balance diversity and reproducibility, generation is staged:

- **Seeds:** We use PostgreSQL- and Oracle-oriented templates, affecting syntax and tokenization. Numeric seed tokens are remapped to stable string prefixes to align WAF and application parameter tracking.
- **Learned Rules:** The harness injects deployment-specific variants, biasing the search towards historically informative outcomes.
- **Transforms:** An optional second layer applies further evasion transforms subject to configuration limits. Consequently, the empirical distribution reflects practical search limits rather than a uniform draw from all possible SQLi strings.

D. Outcome Semantics and Metrics

Each attempt i yields an HTTP status s_i and body b_i, and is labeled $y_i \in \{clean_bypass, error_bypass, blocked, other\}$. A blocked label denotes edge denial. A bypass label indicates a WAF-pass with application-visible signals consistent with effective SQLi side effects. The "other" label covers ambiguous outcomes.

An attempt i is considered as $y_i = \text{clean_bypass}$ if the HTTP response status code is either 200 OK which indicates a clean bypass reaching the application logic; $y_i = \text{error_bypass}$ if the status code is 500 Internal Server Error indicating that the mutated payload successfully bypassed the WAF edge and caused an unhandled database or ORM exception at the application layer.

Let N be the total attempts and N_{bp} the bypass count. The aggregate bypass rate is:

$$\text{BR}_{\text{agg}} = \frac{N_{\text{bp}}}{N} \tag{2}$$

For a mutation family f, the family-level bypass rate is $\text{BR}_f = N_{f,\text{bp}}/N_f$. We also measure unique payload cardinality to identify whether high attempt-level rates result from diverse payloads or string reuse.

E. Secondary Control

To assess normalization effects, we applied fixpoint `unquote_plus` decoding to probe strings and trained a TF-IDF logistic-regression surrogate (`ml_baseline.py`) on 20 labeled HTTP templates. This pilot tests whether minimal RFC-relevant decoding aligns feature spaces sufficiently to improve a simple classifier.

IV. Experiments and Results

We answered RQ1–RQ3 using data exported from live AWS WAF probing and an RFC decoding surrogate pilot. Scripts were executed in a Windows environment using standard data science libraries (scikit-learn, joblib).

A. Experimental Flow

We followed a consistent workflow:

1) Start with SQLi seeds and remap IDs to the target format.
2) Apply mutation rules and send GET requests to the endpoint.
3) Classify responses as bypass, blocked, or other.
4) Aggregate outcomes to compare family-level behavior.

B. Overall Summary

Table I presents the high-level outcome distribution across the 21,000 tested payloads.

C. Status Code Breakdown

Table II shows the distribution of HTTP status codes observed.

D. Transformation Effectiveness

Table III summarizes bypass and block counts per transformation type, derived from `learned_encodings.json`.

TABLE I
Overall result distribution

Result Type	Count	Percentage
Bypass (200 OK and 500)	15,457	73.6%
Blocked by WAF (403)	3,221	15.3%
Other (404 Not Found)	2,322	11.1%

TABLE II
HTTP status distribution

Status Code	Count	Description
200 OK	9,866	Successful request (Clean application pass)
403 Forbidden	3,221	Request blocked by AWS WAF edge policy
404 Not Found	2,322	Resource not found (Payload parsing anomaly)
500 Internal Server Error	5,591	Server error (often due to injection)

E. Key Takeaways

- Double/triple URL-encoding and null-byte suffixes are the most effective techniques to bypass the WAF.
- Transformations that replace spaces with "+", introduce tab/newline characters, or inject comment blocks are largely blocked.
- Server-side double-decoding has been mitigated, reducing 500 errors during testing.

F. RQ1: Scale, Labels, and Unique Payloads

The new experimental run tested $N = 21{,}000$ payload variants, achieving $N_{\text{bp}} = 15{,}457$ successful bypasses (73.6%) and $N_{\text{blk}} = 3{,}221$ blocked attempts (15.3%). The remaining $2{,}322$ responses were other outcomes such as 404 or server errors (11.1%). Table IV summarizes these aggregate statistics.

G. RQ2: Family-Level Heterogeneity

Bypass success is highly concentrated within specific mutation categories. Figure 2 and Table V demonstrate a stark polarization in evasion efficacy. The null_byte family achieves a near-absolute bypass rate ($\text{BR}_{\text{f}} = 99.9\%$), while the large-scale url_encode family maintains a high yield of 72.3%, serving as the primary driver of successful edge evasions by volume. Conversely, structural mutations such as space_modifiers are completely suppressed ($\text{BR}_f = 0\%$) by the AWS WAF edge filters. This pattern indicates that policy stress is heavily concentrated in specific syntactic strategies rather than being uniformly distributed across all transformation types.

url_encode and null_byte yield high success rates, while space_modifiers are systematically suppressed ($N = 21{,}000$).

Fig. 2. Contrasting family-level bypass behavior.

TABLE III
BYPASS/BLOCK STATISTICS BY TRANSFORM

Transform	Bypass	Blocked	Other	Reach	Success Rate (%)
url_encode	8,534	3,274	0	11,808	72.3%
null_byte	7,209	0	2	7,211	99.9%
case_mix	97	6	2,224	2,327	4.2%
other	88	12	1	101	87.1%
hex	10	1	32	43	23.3%
tab	1	6	0	7	14.3%
comment_inline	1	2	0	3	33.3%

Note. Reach denotes the total number of payload variants containing a given mutation style. Categories are not mutually exclusive because multiple mutation styles may be compounded within a single payload.

TABLE IV
AGGREGATE RESULTS OF ML-WAF AWS PROBING (21K VARIANTS).

Metric	Count	Percentage
Variants tested N	21,000	100%
Bypass count N_{bp}	15,457	73.6%
Blocked count N_{blk}	3,221	15.3%
Other outcomes N_{oth}	2,322	11.1%

H. Threats to Validity and Reproducibility

Threats. Our labels rely on script-specific heuristics. Results are bound to a specific policy snapshot and single-channel URL injection. Furthermore, the search distribution is explicitly non-uniform, reflecting operational biases. **Reproducibility.** Summaries can be regenerated with scripts in the repository. Executing `run_probe.py` yields fresh logs, though counts will naturally drift as WAF policies evolve. Probing was conducted exclusively against authorized targets.

V. DISCUSSION AND CONCLUSION

This work decomposes SQLi WAF bypass analysis along semantics, scale, and structure under an induced search distribution.

A. Research Question Answers

RQ1 (Scale and semantics): Our comprehensive live execution with ($N = 21{,}000$) payload variants revealed an aggregate bypass rate ($\mathrm{BR}_{\mathrm{agg}}$) of 73.6%. The high volume of successful edge evasions is heavily driven by multi-layered encoding strategies that successfully bypass the Application Load Balancer (ALB) WAF rulesets but trigger structural anomalies down the line.

RQ2 (Structure): Evasion efficacy is intensely polarized across mutation categories. The null_byte suffix family achieved an absolute bypass rate of 99.9% (7,209 successful bypasses out of 7,211 attempts) under the fixed AWS policy configuration. Conversely, space substitution primitives (plus_space, newline, vtab) and structured comment_block modifications were completely mitigated ($\mathrm{BR}_{\mathrm{f}} = 0\%$), proving that the WAF's lexical tokenization filters are highly optimized for space-boundary semantics but weak against encoding anomalies.

TABLE V
SELECTED MUTATION FAMILIES: ATTEMPTS N_f AND BR_f.

Family Name	Reach	BR_f (%)	Role
url_encode	11,808	72.3%	High Scale / Core Driver
null_byte	7,211	99.9%	Maximum Yield Evasion
space_modifiers	> 0	0.0%	Systematically Suppressed

B. Implications and Conclusion

For defenders, policy tuning and regression testing must be family-aware; relying on aggregate dashboards is insufficient. Edge decisions should be combined with robust application-layer controls. For offensive researchers, publishing induced search configurations and explicit outcome logs is vital for reproducible science.

We presented a reproducible black-box methodology for SQLi WAF evaluation. The core takeaway is that meaningful offensive evaluation demands decomposed statistics and transparent search bias. Our family-level analysis identifies specific operational vulnerabilities, reinforcing the need for layered, context-aware defenses.

Limitations and Ethics. Our findings are bounded by a single policy snapshot and URL injection channel. All methodology is strictly intended for authorized testing environments.

ACKNOWLEDGMENT

The authors thank the faculty and staff of the School of Information and Communication Technology, Hanoi University of Science and Technology, for their academic guidance and support throughout this research. The authors also appreciate the constructive feedback from colleagues and peers, which helped improve the quality of this work.

REFERENCES

[1] Y. Guan et al., "SSQLi: A Black-Box Adversarial Attack Method for SQL Injection Based on Reinforcement Learning," *Future Internet*, vol. 15, no. 4, p. 133, 2023.

[2] S. A. Akhavani et al., "WAFFLED: Exploiting Parsing Discrepancies to Bypass Web Application Firewalls," in *Proc. Security Research Workshop*, 2025.

[3] S. A. Akhavani et al., "HTTP-Normalizer: A robust proxy tool for RFC-compliant HTTP validation," in *Proc. Security Research Workshop*, 2025.

[4] B. Zhang et al., "BWAFSQLi: Bypassing Web Application Firewall with Adversarial SQL Injections," *ACM Trans. Softw. Eng. Methodol.*, 2026.

[5] C. Wu et al., "WAFBOOSTER: Automatic Boosting of WAF Security Against Mutated Malicious Payloads," in *Proc. Int. Conf. Cybersecurity*, 2025.

[6] L. Demetrio et al., "WAF-A-MoLE: Evading Web Application Firewalls through Adversarial Machine Learning," in *Proc. 35th ACM/SIGAPP Symp. Appl. Comput. (SAC)*, 2020.

[7] L. M. Khan et al., "GSQLi: A GAN-based Approach for Adversarial SQL Injection Sample Generation against WAF," in *Proc. Int. Conf. Inf. Secur.*, 2024.

[8] H. O. E. Elebiary et al., "Enhanced Web Payload Classification Using WAMM: An AI-Based Framework for Dataset Refinement and Model Evaluation," *J. Inf. Secur. Appl.*, 2026.

[9] OWASP, "Core Rule Set (CRS) Documentation," OWASP Foundation, 2020.

[10] R. Fielding et al., "RFC 7230: Hypertext Transfer Protocol (HTTP/1.1): Message Syntax and Routing," IETF, 2014.

[11] T. Berners-Lee et al., "RFC 3986: Uniform Resource Identifier (URI): Generic Syntax," IETF, 2005.